



MAGIC Grants Monero FCMP++ Crypto

Security Assessment

July 15, 2026

Prepared for:

Justin Ehrenhofer

MAGIC Grants

Prepared by: **Joe Doyle**

Table of Contents

Table of Contents	1
Project Summary	2
Executive Summary	3
Project Goals	6
Project Targets	7
Project Coverage	8
Automated Testing	9
Summary of Findings	10
Detailed Findings	11
1. Edwards-to-Weierstrass point conversion computes an incorrect field inverse but a correct overall result for (0,-1)	11
2. ge_double_scalarmult functions leave output uninitialized when scalars are zero	13
3. fe_equals may not be constant-time	15
4. Edwards-to-Weierstrass conversion outputs negated y-coordinate	16
5. sc_check left-shifts negative values	17
6. Rust FFI function selene_scalar_from_bytes depends on fragile memory layout assumptions	18
A. Vulnerability Categories	20
B. Fix Review Results	22
Detailed Fix Review Results	23
C. Fix Review Status Categories	24
About Trail of Bits	25
Notices and Remarks	26

Project Summary

Contact Information

The following project manager was associated with this project:

Mary O'Brien, Project Manager
mary.obrien@trailofbits.com

The following engineering director was associated with this project:

Jim Miller, Engineering Director, Application Security and Cryptography
james.miller@trailofbits.com

The following consultants were associated with this project:

Joe Doyle, Consultant
joseph.doyle@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
May 07, 2026	Pre-project kickoff call
May 15, 2026	Status update meeting #1
May 22, 2026	Delivery of report draft
May 22, 2026	Report readout meeting
July 15, 2026	Delivery of final comprehensive report with fix review

Executive Summary

Engagement Overview

The MAGIC Grants community fund engaged Trail of Bits to review the security of the Monero FCMP++ Cryptography implementation.

These changes include several incremental modifications to the Monero cryptographic core libraries, which lay the groundwork for the Full-Chain Membership Proofs migration.

A team of one consultant conducted the review from May 11 to May 22, 2026, for a total of two engineer-weeks of effort. Our testing efforts focused on correctness and security of each change. With full access to source code and documentation, we performed static and dynamic testing of the codebase, using automated and manual processes.

This review focused on the “1a” and “1b” phases of the FCMP++ Integration Audit plan, in particular:

- Zero commitment generation function `zeroCommitVartime`
- Field arithmetic functions including `fe_batch_invert`, `fe_equals`, and `fe_reduce_vartime`
- Curve25519 torsion clearing logic
- The Selene Rust/C FFI interface
- Functions converting between curve25519 coordinate representations
- Functions for converting output records into leaf tuples.
- The “unbiased key image generator” hash-to-curve implementation
- Output conversion for FCMP++ migration

Observations and Impact

The changes made are focused and appear to be correct. We found six informational findings, including narrow cases of undefined behavior ([TOB-MAGIC-FCMP-2](#), [TOB-MAGIC-FCMP-5](#)) and violated preconditions ([TOB-MAGIC-FCMP-1](#)). However, none appear to lead to incorrect results or exploitable behavior within the Monero system.

In addition to the findings listed in this report, we confirmed that the key image generator is an implementation of the Elligator 2 hash-to-curve algorithm, and the “unbiased” version implements the “hash-twice-and-add” scheme.

We also confirmed that the conversion performed for insertion into the FCMP++ tree prevents existing spendable records with torsion from aliasing to the same leaf in the tree,

and that, according to the spending preconditions provided by the Monero team, no currently spendable record will be rendered unspendable by the FCMP++ migration.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that Monero take the following steps before deploying the FCMP++ migration:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Expand the test suite with additional edge-case tests, known-answer tests, and sanitizers such as UBSan and Miri.** While tests were present, edge case values such as 0 were not tested in helper functions ([TOB-MAGIC-FCMP-1](#), [TOB-MAGIC-FCMP-2](#)), the coordinate conversion is slightly incompatible with other libraries ([TOB-MAGIC-FCMP-4](#)), and UBSan flags a widely-used function ([TOB-MAGIC-FCMP-5](#)).
- **Define an explicit, well-defined serialization boundary for Rust-C FFI.** The current Rust-C interfaces are fragile, and for most types that would be passed between languages, there are efficient serialization formats that can definitively avoid cross-language quirks.
- **Document the exact historical spendable-record requirements.** The migration of existing records to FCMP++ clears public keys and output commitments of torsion components, and requires that the resulting torsion-cleared values are not the identity point. According to the Monero team, any record that would be rejected by these checks is already impossible to spend based on existing consensus rules. As described below, we have confirmed that these rules are sufficient, but have not confirmed that every transaction-validation code path enforces these precise rules.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

<i>Severity</i>	<i>Count</i>
High	0
Medium	0
Low	0
Informational	6
Undetermined	0

CATEGORY BREAKDOWN

<i>Category</i>	<i>Count</i>
Cryptography	2
Data Exposure	1
Undefined Behavior	3

Project Goals

The engagement was scoped to provide a security assessment of the Monero FCMP++ Crypto Integration. Specifically, we sought to answer the following non-exhaustive list of questions:

- Are all the new cryptographic primitive implementations faithfully and securely implementing their intended primitives? This includes:
 - Field element operations
 - Torsion clearing
 - Elliptic curve coordinate system changes
 - A uniform-output variant of the Elligator 2 hash-to-curve
- Is the Rust-C FFI function `selene_scalar_from_bytes` correct and safe?
- Are there any historical output formats that could be rendered unspendable by the FCMP++ migration?

Project Targets

The engagement involved reviewing and testing the following target.

monero

Repository	https://github.com/monero-project/monero
Version	86a0aa46f59f53a1a96a4e239b9663e4ccd3e30d, PR #10360
Type	C++, Rust
Platform	Native

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- Manual review of the implementation
- Review of specification documents and relevant publications such as the [IETF draft specification](#) for curve representations and [the Elligator paper](#)
- Static analysis of the implementation
- Manual review of receive-detection algorithms

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- We were not able to comprehensively determine whether the rules for a valid spendable transaction record provided by the Monero team match the precise transaction-validation methods currently present in Monero. It is possible that a pathological transaction record may be currently spendable due to a validation edge case not covered by those rules, and thus would not be migratable to FCMP++.
- We did not comprehensively evaluate historical or out-of-tree methods for receive-detection and transaction validation. If a record was incorrectly marked spendable or if previous transaction validation changes rendered it unspendable, we considered that out of scope for the FCMP++ migration.

Automated Testing

Trail of Bits uses automated techniques to extensively test the security properties of software. We use both open-source static analysis and fuzzing utilities, along with tools developed in-house, to perform automated testing of source code and compiled software.

Test Harness Configuration

We used the following tools in the automated testing phase of this project:

Tool	Description	Policy
Semgrep	An open-source static analysis tool for finding bugs and enforcing code standards when editing or committing code and during build time	Appendix C

Areas of Focus

Our automated testing and verification work focused on the following system properties:

- General code quality issues and unidiomatic code patterns

Test Results

The results of this focused testing are detailed below.

Semgrep

We ran Semgrep Pro on the codebase using both the default configuration and our internal rulesets. This analysis did not directly identify any security issues in the codebase.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Edwards-to-Weierstrass point conversion computes an incorrect field inverse but a correct overall result for (0,-1)	Cryptography	Informational
2	ge_double_scalarmult functions leave output uninitialized when scalars are zero	Undefined Behavior	Informational
3	fe_equals may not be constant-time	Data Exposure	Informational
4	Edwards-to-Weierstrass conversion outputs negated y-coordinate	Cryptography	Informational
5	sc_check left-shifts negative values	Undefined Behavior	Informational
6	Rust FFI function selene_scalar_from_bytes depends on fragile memory layout assumptions	Undefined Behavior	Informational

Detailed Findings

1. Edwards-to-Weierstrass point conversion computes an incorrect field inverse but a correct overall result for (0,-1)

Severity: Informational

Difficulty: Not Applicable

Type: Cryptography

Finding ID: TOB-MAGIC-FCMP-1

Target: `src/fcmp_pp/fcmp_pp_crypto.cpp`

Description

Curve25519 curve points are converted from Edwards coordinates to Weierstrass coordinates via the `point_to_wei_x_y` function, which calls `point_to_ed_derivatives`, shown below in figures 1.1–1.3.

```
bool point_to_wei_x_y(const crypto::ec_point &pub, crypto::ec_coord &wei_x,
crypto::ec_coord &wei_y) {
    EdDerivatives ed_derivatives;
    if (!point_to_ed_derivatives(pub, ed_derivatives))
        return false;
    return ed_derivatives_to_wei_x_y(ed_derivatives, wei_x, wei_y);
}
```

Figure 1.1: `monero/src/fcmp_pp/fcmp_pp_crypto.cpp#108–113`

```
bool point_to_ed_derivatives(const crypto::ec_point &pub, EdDerivatives
&ed_derivatives) {
    if (pub == crypto::EC_I)
        return false;
    // fe y;
    ge_p3 p3;
    if (ge_frombytes_vartime(&p3, to_bytes(pub)) != 0)
        return false;
    fe one;
    fe_1(one);
    // (1+y), (1-y)
    fe_add(ed_derivatives.one_plus_y, one, p3.Y);
    fe_sub(ed_derivatives.one_minus_y, one, p3.Y);
    // (1-y) * x
    fe_mul(ed_derivatives.one_minus_y_mul_x, ed_derivatives.one_minus_y, p3.X);
    return true;
}
```

Figure 1.2: `monero/src/fcmp_pp/fcmp_pp_crypto.cpp#69–84`

```

bool ed_derivatives_to_wei_x_y(const EdDerivatives &ed_derivatives, crypto::ec_coord
&wei_x, crypto::ec_coord &wei_y) {
    static const int N_ELEMS = 2;

    // Get inverse of (1-y) and ((1-y)*x)
    std::unique_ptr<fe[]> fe_batch = std::make_unique<fe[]>(N_ELEMS);
    std::unique_ptr<fe[]> inv_res = std::make_unique<fe[]>(N_ELEMS);

    memcpy(&fe_batch[0], &ed_derivatives.one_minus_y, sizeof(fe));
    memcpy(&fe_batch[1], &ed_derivatives.one_minus_y_mul_x, sizeof(fe));

    fe_batch_invert(inv_res.get(), fe_batch.get(), N_ELEMS);

    fe_ed_derivatives_to_wei_x_y(
        to_bytes(wei_x),
        to_bytes(wei_y),
        inv_res[0]/*(1/(1-y))*/,
        ed_derivatives.one_plus_y,
        inv_res[1]/*(1/((1-y)*x))*/);

    return true;
}

```

*Figure 1.3: If $(1-y)*x$ is zero, batch inversion results in $[0, 0]$.
([monero/src/fcmp_pp/fcmp_pp_crypto.cpp#86-106](#))*

Since the formula involves inverting $(1-y)$, there is a check to exclude the point $(0, 1)$, but the point $(0, -1)$ is not excluded. Since $x=0$ in that case, the batch inversion of $[2, 0]$ yields the array $[0, 0]$ instead of $[1/2, 0]$. However, this still yields the correct point $(A/3, 0)$ in Weierstrass coordinates.

Recommendations

Short term, document this behavior, or strengthen the input validation to reject any point where $x=0$ or $y=1$ instead of only rejecting the identity point.

Long term, ensure that all cryptographic functions have documented input assumptions that are thoroughly tested.

2. ge_double_scalarmult functions leave output uninitialized when scalars are zero

Severity: Informational

Difficulty: Not Applicable

Type: Undefined Behavior

Finding ID: TOB-MAGIC-FCMP-2

Target: src/crypto/crypto-ops.c

Description

The functions `ge_double_scalarmult_base_vartime_p3` and `ge_double_scalarmult_precomp_vartime2_p3` each take an output parameter `r3`, which may be uninitialized. Each of them follows the pattern shown below in figure 2.1, where the loop counter `i` is initialized to 255, and decremented until the highest nonzero bit of either `a` or `b` is found; only then does it perform the main loop. The main loop then writes to `r3` only when `i == 0`, but the loop is never entered when the scalars are 0, because `i` will be -1.

```
void ge_double_scalarmult_base_vartime_p3(ge_p3 *r3, const unsigned char *a, const
ge_p3 *A, const unsigned char *b) {
    signed char aslide[256];
    signed char bslide[256];
    ge_dsmp Ai; /* A, 3A, 5A, 7A, 9A, 11A, 13A, 15A */
    ge_p1p1 t;
    ge_p3 u;
    ge_p2 r;
    int i;

    slide(aslide, a);
    slide(bslide, b);
    ge_dsm_precomp(Ai, A);

    ge_p2_0(&r);

    for (i = 255; i >= 0; --i) {
        if (aslide[i] || bslide[i]) break;
    }

    for (; i >= 0; --i) {
        ge_p2_dbl(&t, &r);

        ...

        if (i == 0)
            ge_p1p1_to_p3(r3, &t);
        else
```

```
    ge_p1p1_to_p2(&r, &t);  
}  
}
```

*Figure 2.1: r3 will never be written if $i = -1$ before the second loop.
([monero/src/crypto/crypto-ops.c#1335-1378](#))*

This function does not appear to be used outside of a client-side proving routine that generates a and b randomly, and thus cannot exercise this undefined behavior.

Recommendations

Short term, move the final write to r3 out of the main loop, or specially handle the $i == -1$ case.

Long term, ensure that all outputs are default-initialized unless the performance impact is prohibitive.

3. fe_equals may not be constant-time

Severity: Informational

Difficulty: Not Applicable

Type: Data Exposure

Finding ID: TOB-MAGIC-FCMP-3

Target: src/crypto/crypto-ops.c

Description

The `fe_equals` function compares two field elements by doing byte-wise `==` comparisons and accumulating them into a flag `r` that is then returned, as shown below in figure 3.1.

```
// return 1 if a == b, else 0
int fe_equals(const fe a, const fe b) {
    unsigned char a_bytes[32];
    unsigned char b_bytes[32];
    fe_tobytes(a_bytes, a);
    fe_tobytes(b_bytes, b);

    int r = 1;
    for (int i = 0; i < 32; ++i) {
        r &= a_bytes[i] == b_bytes[i];
    }
    return r;
}
```

*Figure 3.1: Use of potentially variable-time comparison
(monero/src/crypto/crypto-ops.c#315-327)*

This function may be compiled to early-exit if one of the comparisons returns false, and the `==` comparison operator may not be constant-time itself. We are not aware of a current compiler and platform that leads to a non-constant-time result.

Recommendations

Short term, adopt an implementation which matches a widely used constant-time-comparison pattern (e.g., the `CRYPTO_memcmp` implementation from [OpenSSL](#)).

Long term, ensure that operations that handle potentially-secret data use common constant-time patterns to prevent compilers from introducing timing side channels.

4. Edwards-to-Weierstrass conversion outputs negated y-coordinate

Severity: Informational

Difficulty: Not Applicable

Type: Cryptography

Finding ID: TOB-MAGIC-FCMP-4

Target: `src/crypto/crypto-ops-data.c`

Description

The conversion between Edwards and Weierstrass coordinates uses the constant `fe_c`, which is supposed to be $\sqrt{-(A+2)}$. The [IETF draft specification](#) for converting to Weierstrass form chooses one of the two square roots, in particular the hex value `0x70d9120b 9f5ff944 2d84f723 fc03b081 3a5e2c2e b482e57d 3391fb55 00ba81e7`. However, the `fe_c` value in the codebase is its negative, `0x0f26edf4 60a006bb d27b08dc 03fc4f7e c5a1d3d1 4b7d1a82 cc6e04aa ff457e06`. Since this appears to be used consistently, it does not directly affect correctness, but may lead to incompatibility with other Curve25519-related codebases.

Recommendations

Short term, determine whether compliance with the de facto standard is required for this codebase; document that decision, and ensure that the test suite reflects that decision.

Long term, ensure that all cryptographic primitives use known-answer tests to ensure compliance with any relevant standards.

5. sc_check left-shifts negative values

Severity: Informational

Difficulty: Not Applicable

Type: Undefined Behavior

Finding ID: TOB-MAGIC-FCMP-5

Target: src/crypto/crypto-ops.c

Description

The `sc_check` function checks whether a scalar is below the Curve25519 group order. The implementation is shown below in figure 5.1.

```
int sc_check(const unsigned char *s) {
    int64_t s0 = load_4(s);
    int64_t s1 = load_4(s + 4);
    int64_t s2 = load_4(s + 8);
    int64_t s3 = load_4(s + 12);
    int64_t s4 = load_4(s + 16);
    int64_t s5 = load_4(s + 20);
    int64_t s6 = load_4(s + 24);
    int64_t s7 = load_4(s + 28);
    return (signum(1559614444 - s0) + (signum(1477600026 - s1) << 1) +
    (signum(2734136534 - s2) << 2) + (signum(350157278 - s3) << 3) + (signum(-s4) << 4)
    + (signum(-s5) << 5) + (signum(-s6) << 6) + (signum(268435456 - s7) << 7)) >> 8;
}
```

Figure 5.1: the `sc_check` implementation ([monero/src/crypto/crypto-ops.c#3868-3878](#))

This implementation generates several values which are either 1, 0, or -1, shifts by various amounts, and adds them together. The C standard specifically states that left-shifting a negative value is undefined behavior. In practice, we are not aware of a common compiler that exploits this, but it does trigger under UBSan.

This appears to already be fixed in master via a `signed_lshift` function.

Recommendations

Short term, fix this implementation to avoid triggering this undefined behavior.

Long term, ensure that the test suite is run with UBSan to detect undefined behavior in the codebase.

6. Rust FFI function selene_scalar_from_bytes depends on fragile memory layout assumptions

Severity: Informational

Difficulty: Not Applicable

Type: Undefined Behavior

Finding ID: TOB-MAGIC-FCMP-6

Target: src/fcmp_pp/fcmp_pp_rust/src/lib.rs

Description

The current FFI exposure of the Rust SeleneScalar type into C and C++ is a single function and struct, shown below in figures 6.1 and 6.2.

```
/// A constant-time implementation of the Ed25519 field.
struct SeleneScalar {
    uintptr_t _0[32 / sizeof(uintptr_t)];
};
```

Figure 6.1: The opaque SeleneScalar type
(monero/src/fcmp_pp/fcmp_pp_rust/fcmp++.h#36-39)

```
macro_rules! ec_elem_from_bytes {
    ($fn_name:ident, $Type:ty, $Curve:ty, $from_bytes:ident) => {
        /// # Safety
        ///
        /// This function assumes 32 bytes allocated, and to be passed a raw pointer
        to
        /// the expected type.
        #[allow(clippy::not_unsafe_ptr_arg_deref)]
        #[no_mangle]
        pub unsafe extern "C" fn $fn_name(bytes: *const u8, ec_elem_out: *mut $Type)
        -> c_int {
            if ec_elem_out.is_null() {
                return -1;
            }
            let mut bytes = unsafe { core::slice::from_raw_parts(bytes, 32) };
            match <$Curve>::$from_bytes(&mut bytes) {
                Ok(ec_elem) => {
                    *ec_elem_out = ec_elem;
                    0
                }
                Err(_) => -2,
            }
        }
    }
};
```

```
ec_elem_from_bytes!(selene_scalar_from_bytes, SeleneScalar, Selene, read_F);
```

Figure 6.2: the C-to-Rust encoding function
([monero/src/fcmp_pp/fcmp_pp_rust/src/lib.rs#7-31](#))

This interface relies on three core assumptions:

1. The in-memory representation of `SeleneScalar` is exactly 32 bytes.
2. `sizeof(uintptr_t)` is a power of 2.
3. If the `SeleneScalar` value is ever read in C/C++, it must have a stable byte representation.

Assumptions (1) and (3) are not guaranteed in general by Rust, although the PR has a pinned dependency on a fork of the `crypto-bigint` crate that patches certain integer types to have `repr(C)`.

While we believe that the current implementation is sound on the platforms it targets, it is extremely fragile, and the performance benefits are unclear compared to calling conversion functions to and from a `[u8; 32]` array.

Recommendations

Short term, replace the raw write with byte-serialization of the field element.

Long term, prefer doing lightweight serialization and deserialization at FFI boundaries.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

On June 15, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the Monero team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, of the six issues described in this report, Monero has resolved five issues, and has partially resolved one issue. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	Edwards-to-Weierstrass point conversion computes an incorrect field inverse but a correct overall result for (0,-1)	Informational	Resolved
2	ge_double_scalarmult functions leave output uninitialized when scalars are zero	Informational	Resolved
3	fe_equals may not be constant-time	Informational	Resolved
4	Edwards-to-Weierstrass conversion outputs negated y-coordinate	Informational	Resolved
5	sc_check left-shifts negative values	Informational	Resolved
6	Rust FFI function selene_scalar_from_bytes depends on fragile memory layout assumptions	Informational	Partially Resolved

Detailed Fix Review Results

TOB-MAGIC-FCMP-1: Edwards-to-Weierstrass point conversion computes an incorrect field inverse but a correct overall result for (0,-1)

Resolved in [PR #18](#). `fe_batch_invert` now detects a zero element and returns an error status instead of inverting zero, with the caller propagating the failure.

TOB-MAGIC-FCMP-2: `ge_double_scalarmult` functions leave output uninitialized when scalars are zero

Resolved in [PR #22](#). The loop condition was updated to ensure that the main loop executes once when scalars are zero, correctly initializing the output to the identity.

TOB-MAGIC-FCMP-3: `fe_equals` may not be constant-time

Resolved in [PR #18](#). The custom byte-accumulation comparison was replaced with libsodium's `crypto_verify_32` for constant-time comparison.

TOB-MAGIC-FCMP-4: Edwards-to-Weierstrass conversion outputs negated y-coordinate

Resolved in [PR #19](#). The `fe_c` constant was negated to match the IETF standard; the monero-oxide Rust implementation has been updated as well ([commit 0662298](#)).

TOB-MAGIC-FCMP-5: `sc_check` left-shifts negative values

Resolved in [commit c5be4dd](#). A portable `signed_lshift` helper was implemented to handle left shifts of negative values, eliminating undefined behavior.

TOB-MAGIC-FCMP-6: Rust FFI function `selene_scalar_from_bytes` depends on fragile memory layout assumptions

Partially resolved in [PR #20](#). Compile-time static assertions for size and alignment have been added on both C and Rust sides to safeguard the FFI interface, though the interface itself remains fragile. The current implementation acknowledges this, and we encourage the Monero team to improve the robustness of this FFI layer in the future:

```
/// This type is expected to be opaque to the C/C++ side, meaning only the Rust side
/// should read/write
/// its internal representation. We're using a modified crypto-bigint crate for this
/// type so that we
/// can work with points and scalars across the FFI without tons of byte repr
/// conversions.
```

Figure B.2: [monero/src/fcmp_pp/fcmp_pp_rust/fcmp%2B%2B.h#46-48](#)

C. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	Description
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to MAGIC Grants under the terms of the project statement of work and has been made public at MAGIC Grants' request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.